

FLIGHT DATA MANAGER

**DANIELE
TAMBONE**

Contents

1	Introduction	1
1.1	Project Objectives	1
1.2	Scope of the Cloud Systems Extension	1
1.3	Methodological Approach	2
2	System Architecture	2
2.1	Architectural Diagram	2
2.2	Networking and Isolation Strategy	3
2.3	Communication Choices	4
3	User Manager Microservice	4
3.1	Application Core and Domain	4
3.2	Ports	5
3.3	Adapters	5
3.4	Repository: Separating Persistence from Idempotency	5
3.5	Middleware and Context Handling	6
3.6	“At-Most-Once” Implementation (Idempotency)	7
4	Data Collector Microservice	8
4.1	Domain and Core Application	8
4.2	Ports	9
4.3	Adapters	9
4.4	Repository	10
4.5	Worker	10
5	Kafka	11
5.1	Introduction and Role in Architecture	11
5.2	Data Flow and Order Guarantee	11
5.3	Implementation Details: Producer	12
5.4	Implementation Details: Consumer	12
5.5	Resilience Testing and Single-Node Limitations	12
6	Orchestration and Monitoring	13

7	Automated Testing	13
7.1	Two Levels	14
8	Local Deployment: a Self-Hosted GitOps Chain	14
8.1	The Five Layers	14
8.2	Three Workflows, Three Responsibilities	15
8.3	Distributing Images Without a Registry	15
8.4	State and Secrets	15
8.5	Storage	16
9	Cloud Deployment on AWS	16
9.1	What Was Substituted	17
9.2	Federated Identity Instead of Stored Keys	17
9.3	Remote State	17
9.4	Network Design	17
9.5	Data Layer	18
9.6	Where Each Secret Lives	18
9.7	Observability	19
9.8	Constraints Imposed by the Account Plan	19
9.9	Consequences for the Entry Point	19
9.10	Cost Model and Operational Discipline	20
	References	21

1 Introduction

This document illustrates the design choices and implementation details of the *Flight Data Manager* project, a distributed microservices-based system originally developed for the course in Distributed Systems and Big Data, and subsequently extended for the course in Cloud Systems (A.Y. 2025-2026).

The goal of the project is to build a scalable and resilient platform for user management and real-time monitoring of air traffic, based on data provided by the public API of the OpenSky Network [1].

1.1 Project Objectives

The system was designed to meet strict functional and non-functional requirements typical of modern cloud-native architectures:

- **Microservices Decomposition:** The application domain was divided into two autonomous services, *User Manager* and *Data Collector*.
- **Resilience and Robustness:** Implementation of failure-handling patterns, including *At-Most-Once* messaging policies, Retry, and Circuit Breaker.
- **Security:** Secure credential management, network isolation (Micro-segmentation), and cross-service authorization checks.
- **Polyglot Persistence:** Use of three databases, each chosen for the shape of the data it holds: *PostgreSQL* for user records, *Redis* for short-lived idempotency keys, and *Neo4j* for modelling relationships between users, interests and flights.
- **Reproducible Deployment:** The entire infrastructure is described as code and provisioned by automated pipelines, on two independent targets — a local Kubernetes cluster and Amazon Web Services.

1.2 Scope of the Cloud Systems Extension

The work carried out for the second course did not alter the application domain. It addressed three areas:

- **A revision of the persistence layer**, separating user records from idempotency tracking within the *User Manager* (Section 3).
- **A local GitOps deployment**, in which a self-hosted Git server drives the provisioning of virtual machines, the bootstrapping of a Kubernetes cluster and the rollout of the application (Section 8).

- **A cloud deployment on AWS**, replacing the self-managed components with managed services and the static credentials with federated identity (Section 9).

The two deployment targets are maintained side by side in the same repository and are deliberately *not* equivalent: the comparison between what must be operated by hand and what can be delegated to a provider is itself one of the results of this work.

1.3 Methodological Approach

Software development was guided by the principles of **Hexagonal Architecture** (or *Ports and Adapters*). This choice allowed us to isolate the *Business Logic* (the Core) from infrastructural details such as databases, HTTP frameworks, or communication protocols.

Such decoupling enables code testability and system maintainability, allowing adapters to be modified without impacting domain rules. This modularity was extended to the infrastructure level: the entire system was containerized using **Docker** to ensure maximum portability.

2 System Architecture

2.1 Architectural Diagram

The system is composed of several containers orchestrated through Docker Compose during development, and by Kubernetes in both deployment targets. Figure 1 illustrates the components and their interactions.

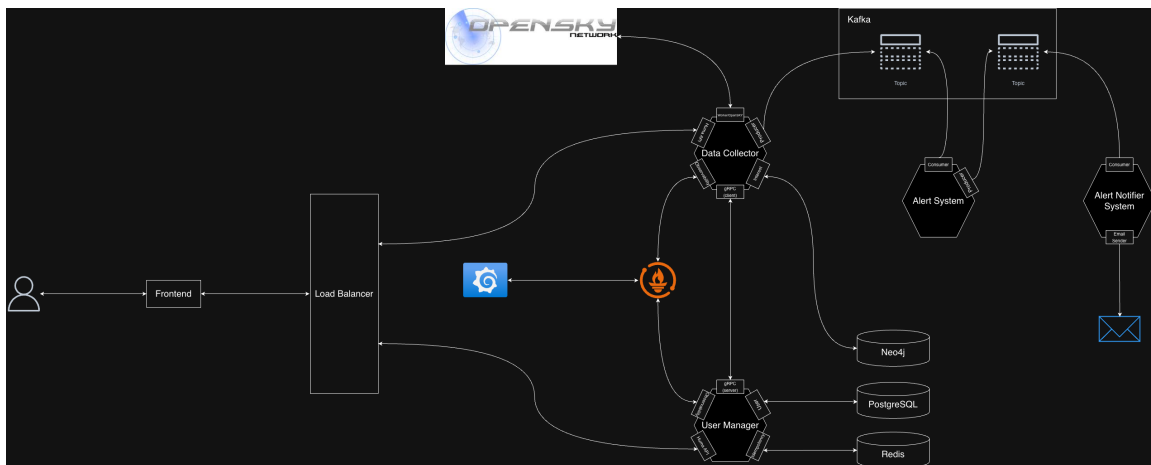


Figure 1: Architectural diagram of the Flight Data Manager system.

- **Frontend:** A single-page application written in *Svelte* and built with *Vite*, served as static assets by an *Nginx* container. It consumes the same REST APIs exposed by the Gateway, addressing them through relative paths so that page and API always share one origin.

- **Restish:** A CLI client used during development and testing to exercise the REST APIs directly, without going through the frontend.
- **External Service:** Public API of the OpenSky Network.
- **API Gateway:** Implemented with *Nginx*, it acts as the single entry point for external traffic, handling SSL termination and routing HTTPS requests to the appropriate microservices (*User Manager* or *Data Collector*).
- **User Manager:** Microservice responsible for identity management. It implements the gRPC service for credential verification and the *At-Most-Once* policy for user registration.
- **Data Collector:** It manages user interests, runs the asynchronous worker to fetch data from the External Service, and acts as a Producer pushing flight data events to the Broker.
- **Broker:** Implemented with *Kafka* (Kraft mode). It manages asynchronous communication via partitioned topics
- **Alert System:** It consumes data from the `to-alert-system` topic, evaluates alert conditions against user interests and produces notification events to the `to-notifier` topic.
- **Alert Notifier System:** The final consumer in the pipeline. It listens to the `to-notifier` topic and handles the delivery of notifications to the end users (e.g., via email adapter).
- **User DB:** Database implemented with *PostgreSQL*, storing user records under a primary key on the e-mail address.
- **Idempotency Store:** Database implemented with *Redis*, holding only the short-lived keys that implement the *At-Most-Once* policy.
- **Data DB:** Database implemented with *Neo4j*, chosen for its native modeling of relationships between users, interests, and flights.

2.2 Networking and Isolation Strategy

One of the key design choices was the adoption of **Micro-segmentation** at the Docker network level to ensure security according to the "Least Privilege" principle. Three distinct networks were defined:

- **gateway-net:** Connects the API Gateway to the two microservices. Allows inbound HTTP traffic.
- **service-net:** A private network dedicated to *inter-service* (gRPC and Kafka) communication.

- **user-net** and **data-net**: Isolated backend networks that connect each microservice exclusively to its own database.

This configuration physically prevents, for example, the *Data Collector* from accessing the users' *Redis* database, thereby reducing the attack surface.

2.3 Communication Choices

2.3.1 External: REST (Huma)

For interaction with clients (end users), REST APIs were exposed. The choice of the **Huma** framework ensured strict input validation (via OpenAPI) and automatic documentation generation.

2.3.2 Internal: gRPC

For synchronous communication between microservices (specifically, when the *Data Collector* needs to verify a user's identity with the *User Manager*), **gRPC** was chosen instead of REST. The motivations for this design choice include:

1. **Performance**: The use of *Protocol Buffers* (binary) reduces latency and data volume compared to JSON, which is essential for frequent internal calls.
2. **Strong Typing**: The contract defined in the `.proto` file ensures that client and server are always aligned on data types, preventing integration errors.
3. **Rich Error Handling**: Adoption of the `google.rpc.Status` standard enables structured and detailed error reporting between services.

3 User Manager Microservice

The **User Manager** is the component responsible for handling the user lifecycle and credential security. It acts as the central identity *Authority* within the distributed system. It is designed to be **stateless**, delegating persistence to two distinct stores: a relational database for user records and an in-memory store for idempotency keys.

3.1 Application Core and Domain

The core of the microservice is defined by the **Domain**, which specifies the **User** entity and its invariant validation rules. To guarantee data consistency, the validation logic is encapsulated directly within the

domain *Factory Function* (**NewUser**). This ensures that a user object can never be instantiated in an invalid state (e.g., malformed email or missing required fields), protecting the system from corrupted data.

The **Application** layer orchestrates the use cases (Registration, Deletion), coordinating data flow between input and output ports without depending on specific technologies.

3.2 Ports

The ports define the interaction contracts of the Core:

- **UserAPI (Driver):** Defines the operations available to clients, including methods for idempotency management.
- **UserRepository (Driven):** Abstracts the persistence of user records, allowing the Core to save and retrieve users while remaining unaware of the underlying database.
- **IdempotencyRepository (Driven):** Abstracts the tracking of in-flight and completed requests. Splitting it from **UserRepository** is what allows the two concerns to be served by different technologies without the Core noticing.

3.3 Adapters

Adapters connect the Core to the outside world:

- **REST Adapter (Huma):** Handles public HTTP requests, validating inputs via OpenAPI schemas and mapping domain errors (e.g., **ErrUserNotFound**) to appropriate HTTP status codes (e.g., 404).
- **gRPC Adapter:** Exposes an interface used by the *Data Collector* to check whether a user exists (**CheckUserExistence**) and to verify credentials (**VerifyCredentials**) with minimal latency.
- **PostgreSQL Repository:** Implements **UserRepository**. User records are stored in a relational table whose primary key on **email** enforces uniqueness at the database level, so a duplicate registration is rejected by the store itself rather than by application logic.
- **Redis Repository:** Implements **IdempotencyRepository**, using the atomic **SETNX** command to claim a request identifier and a 24-hour TTL to expire it afterwards.

3.4 Repository: Separating Persistence from Idempotency

An earlier revision of the system used **Redis** for both concerns: user profiles serialised as JSON under **user:<email>**, and idempotency locks. The two were later separated, because they have opposite requirements.

- **User records** must survive restarts indefinitely, benefit from a schema and from a uniqueness constraint, and are queried by a natural key. These are the properties of a relational store, and **PostgreSQL** provides them.
- **Idempotency keys** are deliberately short-lived: they exist only to recognise a retry within a bounded window, and must *expire on their own*. A TTL is the natural mechanism, and **Redis** provides it together with the atomic **SETNX** needed to prevent concurrent registrations from racing.

Keeping both in a single store forced the worst compromise for each: durability where it was not needed, and no expiry where it was. The separation also has a practical consequence in the cloud deployment described in Section 9, where the two map onto two distinct managed services, *Amazon RDS* and *Amazon ElastiCache*.

The split is directly observable at runtime. Registering the same user twice with the same **X-Request-ID** returns two responses whose **registered_at** timestamps differ in their last three digits: the cached reply preserves nanosecond precision from the original JSON, while a subsequent read from PostgreSQL is truncated to the microsecond precision of the **timestamp_tz** column.

3.5 Middleware and Context Handling

Adopting a microservice architecture with an API Gateway (Nginx) introduces a technical challenge related to identity and network origin: *IP address masking*. Since all incoming traffic passes through the Gateway, the *User Manager* technically sees every request as originating from the internal Nginx container IP (e.g., **172.18.0.x**), losing the actual client IP information.

This is a critical issue for the *Scoped Idempotency* mechanism, which relies on caller IP differentiation for security.

3.5.1 IP Extraction Middleware

To solve this issue, a custom **HTTP Middleware** (**internal/middleware**) was implemented to intercept each request before it reaches the application layer (Huma).

The middleware applies a priority-based extraction strategy to determine the real originator of the request:

1. **X-Forwarded-For header:** The de-facto standard for reverse proxies. If present, the middleware extracts the first IP in the list (the original client), trusting the proxy chain configured in Nginx.
2. **X-Real-IP header:** Used as a fallback when configured by specific load balancers.

3. **RemoteAddr:** Used as a last resort (direct connection) when the service is accessed without passing through the Gateway, such as during local testing or debugging.

3.5.2 Context Injection

Once extracted, the real IP is not passed explicitly as a parameter to controller methods; instead, it is injected into the request's `context.Context`. This design choice, idiomatic in Go, allows the “contextual” information (who is making the request) to propagate transparently through the application layers, making it accessible to the Handler for idempotency hash computation without polluting business-logic function signatures.

3.6 “At-Most-Once” Implementation (Idempotency)

To satisfy the *At-Most-Once* messaging requirement and robustly handle automatic client retries (or network errors during ACK delivery), a **Scoped Idempotency** mechanism was implemented.

3.6.1 The Problem

In a distributed system, if a client sends a registration request and does not receive a response (time-out/network error), it will attempt to send it again. Without idempotency, this could result in:

- Creation of duplicate users.
- Spurious errors (“User already exists”) that confuse the user, who believes the registration failed even though it actually succeeded.

3.6.2 The Solution: Idempotency Key and Data Structure

The client must generate a unique ID for each operation attempt (**X-Request-ID** header). The server uses this ID to track the request state in Redis.

The Redis lock key is built by combining the caller identity and the request ID:

$$Key = \text{idempotency} : \text{SHA256}(IP_{client}) : \text{RequestID} \quad (1)$$

Including the client IP hash prevents accidental collisions among different users who might reuse the same ID.

The value associated with the key is not a simple flag but a JSON structure storing the full operation state:

- **Status:** Indicates the current state (**PROCESSING** or **COMPLETED**).

- **Response:** A serialized copy of the response body (JSON) sent to the client. This allows returning the exact same original response in case of retries.
- **Timestamp:** The time when the first request was received.

3.6.3 States

The registration process follows two states, managed atomically in Redis:

1. **PROCESSING:** As soon as a request with a new ID arrives, the system acquires an atomic *lock* (via `SETNX`) storing the **PROCESSING** state. If duplicate requests arrive while the lock is active, they are rejected (`429 Too Many Requests`) to avoid race conditions.
2. **COMPLETED:** Once processing completes, the Redis record is updated: the state becomes **COMPLETED** and the **Response** is stored. If the client retries with the same ID, the system immediately returns the stored snapshot, simulating an instant success without touching the user database.

In case of a blocking application error during registration (e.g., invalid password), the system performs a **ROLLBACK**, deleting the idempotency key. This restores the initial state, allowing the client to retry the operation with the same ID (e.g., after correcting the parameters).

4 Data Collector Microservice

The **Data Collector** represents the analytical engine of the platform, responsible for aggregating, storing, and retrieving flight data. Unlike the User Manager, which has a purely reactive nature, this microservice adopts a hybrid model that combines synchronous interaction with the user and background processing for database population.

4.1 Domain and Core Application

The core of the service is modeled around two main entities: **Interest** and **Flight**. The **Interest** entity encapsulates the user's intent to monitor a specific airport, including validation logic to ensure ICAO code compliance. The **Flight** entity, on the other hand, acts as a normalized data transfer object (DTO), abstracting away the complexity and heterogeneity of the raw data provided by the external provider OpenSky Network.

The Core orchestrates two distinct yet interconnected workflows. The first is the *user flow*, dedicated to handling real-time requests (interest registration, statistics queries), where each operation is subject to strict credential verification via gRPC. The second is the *worker flow*, a fully decoupled background process responsible for the entire data lifecycle, from acquisition to cleaning.

4.2 Ports

Ports define the boundaries of the system:

- **CollectorAPI (Driver):** Exposes business functionalities, including batch management of interests and statistical queries.
- **FlightRepository (Driven):** Abstracts persistence on the graph database, providing specific methods for relational queries (“The user follows airport X”).
- **ServiceClient (Driven):** Defines contracts for external dependencies (*User Manager* and *OpenSky*).

4.3 Adapters

Adapters connect the Core to the external world, handling both inputs (from users or time) and outputs (towards databases or third-party services):

- **REST Adapter (Huma):** As in the User Manager, it handles the public HTTP interface. In addition to input validation, it manages DTO-based data transformation: it converts internal Unix timestamps into human-readable formats (ISO 8601) and translates domain security errors (e.g., `ErrUserNotAuthorized`) into appropriate HTTP responses (e.g., 401).
- **gRPC Client Adapter:** Manages synchronous communication with the *User Manager*. It implements client-side resilience patterns such as **Retry** (for transient network errors) and **Circuit Breaker** (to prevent cascading failures), making the system robust even in case of User Manager unavailability.
- **Neo4j Repository:** Translates domain operations into *Cypher* queries. It leverages features such as `UNWIND` for batch insertion of interests and `MERGE` to ensure idempotency at the database level, preventing duplicates in the graph.
- **OpenSky Client:** Extracts data via the public OpenSky Network API.
- **Worker Adapter:** A *Driving* adapter acting as a temporal trigger. It decouples time management from business logic, periodically initiating data collection and cleaning processes.
- **Kafka Producer Adapter:** Implements the asynchronous communication channel towards the message broker. It is responsible for publishing events to the `to-alert-system` topic.

4.4 Repository

A *Domain-Specific* persistence strategy was adopted, selecting **Neo4j** as the storage technology. This choice is motivated by the intrinsic topology of the data, which naturally lends itself to a graph representation: users are connected to airports through interest relationships (`INTERESTED_IN`), while flights connect departure and arrival airports (`DEPARTED`, `ARRIVED`).



Figure 2: Database schema.

Using a graph database provides significant performance advantages over traditional relational joins. Additionally, data integrity and consistency are ensured at the query level: the `MERGE` clause makes flight insertion an idempotent operation, preventing duplicates even when data import cycles are re-executed, while the use of `UNWIND` supports efficient batch processing for mass insertion of interests.

4.5 Worker

Data acquisition is handled by a cyclic *Background Worker* (with a configurable frequency, default 12 hours). To optimize resource usage and respect the rate-limiting constraints imposed by external APIs, a *Checkpointing* strategy has been implemented. The system stores, for each airport, the timestamp of the last update: at every cycle, the worker evaluates the “freshness” of the data and, if the update is recent, avoids redundant calls to OpenSky. Otherwise, it proceeds with *Incremental Fetching*, downloading only the missing time window.

5 Kafka

5.1 Introduction and Role in Architecture

In compliance with the project requirements, a Message Broker (Apache Kafka) has been introduced to decouple system components. The architecture evolves from synchronous communication to an **event-driven** one, improving resilience and scalability.

The integration follows the principles of **Hexagonal Architecture**:

- The **Producers** (Driven Adapters) publish events externally.
- The **Consumers** (Driver Adapters) react to incoming events.

The variable `KAFKA_NUM_PARTITIONS: 3` sets the default to 3 partitions for automatically created topics. This configuration prepares the system for horizontal scalability (Parallel Consumer), allowing multiple instances of the same service to process messages in parallel.

5.2 Data Flow and Order Guarantee

The system uses two main topics. Despite the presence of multiple partitions, temporal data integrity is guaranteed by the strict use of **Message Keys**.

5.2.1 Topic to-alert-system

- **Source:** data-collector → **Destination:** alert-system
- **Payload:** AirportUpdateEvent (Arrival/Departure flight).
- **Partition Key (Key):** AirportCode.

Using the airport code as a key ensures that all updates related to the same airport (e.g., "LIRF") are always routed to the same partition (thanks to consistent hashing). This guarantees that the **alert-system** receives chronological updates in the exact order of generation, avoiding race conditions on timestamps.

5.2.2 Topic to-notifier

- **Source:** alert-system → **Destination:** alert-notifier-system
- **Payload:** Notification (Email).
- **Partition Key (Key):** UserEmail.

By using the user's email as a key, we guarantee that all notifications directed to a specific person are handled sequentially, preventing the arrival of disordered alerts.

5.3 Implementation Details: Producer

Producers are configured for maximum reliability (`acks: "all", retries: 3`).

- **Asynchronous Delivery:** A dedicated goroutine (`deliveryReportHandler`) monitors the sending outcome without blocking the main thread.
- **Throughput Optimization:** `linger.ms: 10` introduces a slight artificial delay (10ms) before sending a batch. This forces the producer to wait for more messages to arrive, resulting in larger batches and fewer network requests.
- **Strict Ordering:** The `max.in.flight.requests.per.connection: 1` configuration is fundamental: in case of a retry due to a network error, it prevents a subsequent message from being written before a previous one, maintaining order even in the face of transient failures.

5.4 Implementation Details: Consumer

Consumers adopt an "At-Least-Once" approach with manual commit management.

- **Polling & Logic:** The consumer performs continuous polling. Upon receipt, the message is deserialized and passed to the business logic (Core).
- **Manual Commit:** The `CommitMessage` method is invoked **only after** the business logic (`CheckThreshold` or `NotifyUser`) has successfully completed. If the service crashes during processing, the message is not committed and will be re-fetched upon restart.
- **Offset Reset:** `auto.offset.reset: "earliest"` ensures that no message is lost if the consumer starts for the first time or if the current offset is invalid.

5.5 Resilience Testing and Single-Node Limitations

During stress testing phases—specifically simulating a Broker outage and subsequent restart during an active ingestion cycle—it was observed that messages buffered in the Producer's internal queue were sometimes rejected upon reconnection with the error: `Unknown topic or partition`.

This behavior is intrinsic to the **Single-Node** configuration deployed in an ephemeral containerized environment. Upon restart, the solitary broker requires a distinct warm-up period to initialize metadata and recover logs. In the absence of an active cluster, there are no peer nodes to maintain topic state or partition leadership during the outage, leading to a temporary unavailability of metadata.

It is important to note that in a real-world production environment, this issue is addressed at the **infrastructure level** rather than the application level guaranteed by deploying a **Kafka Cluster** with a **Replication Factor** of $N > 1$.

6 Orchestration and Monitoring

To enhance scalability and observability, the system infrastructure was evolved from a monolithic-like Docker Compose environment to a distributed **Kubernetes** cluster, introducing a specific white-box monitoring layer.

The cluster used during this phase was initially created with **Kind** (Kubernetes in Docker). It was later replaced by a cluster built with **kubeadm** on dedicated virtual machines, and finally by a managed cluster on AWS; both are described in the deployment sections that follow. The monitoring layer described here was carried over to each of them *without modification*, which is discussed in Section 9.

A monitoring infrastructure based on **Prometheus** was integrated to collect performance data directly from the microservices (White-box monitoring).

A specific Middleware (Decorator pattern) was implemented to intercept HTTP requests and gRPC calls transparently, decoupling the monitoring logic from the core business logic.

The system exposes the following metric types via the `/metrics` endpoint:

1. **Gauge** (`_duration_seconds`): Used to measure the latency of operations.
2. **Counter** (`..._requests_total`): Used to track throughput and error rates. It counts the total number of requests received, strictly categorized by HTTP status codes.

To provide granular observability, every metric is enriched with specific labels:

- **Infrastructure Labels:** `service` (application name) and `node` (the Kubernetes node hosting the pod).
- **Operational Labels:** `method` (GET/POST), `endpoint` (API path), and `status` (HTTP response code).

This setup allows for precise queries, such as identifying if a specific node is underperforming or if a specific endpoint is generating a high rate of errors.

7 Automated Testing

The test suite is deliberately shallow in quantity and precise in placement: it covers the two boundaries where a regression would be silent, rather than aiming at a coverage figure.

7.1 Two Levels

- **Unit tests on the Core.** The domain and the application layer are tested against in-memory implementations of their ports. Because the Core depends only on interfaces, no database, broker or HTTP server is involved, and the tests describe business rules rather than infrastructure. The registration flow, the rejection of duplicate users and the verification of a wrong password are all exercised this way.
- **Component-integration tests on the adapters.** Each microservice is tested at its public boundary, with the transport actually running. The HTTP adapters of *User Manager* and *Data Collector* are exercised through `httptest` against a real Huma router, so that OpenAPI validation and error mapping are covered. The gRPC adapter is tested over `bufconn`, an in-memory listener that runs the real gRPC stack without opening a port. The Kafka consumers are tested by feeding them the payloads they would receive from the broker.

The distinction matters because the two levels fail for different reasons. A unit test breaks when a business rule changes; an adapter test breaks when a contract does — a renamed field, a status code that no longer matches, a protobuf message out of step with its consumer.

Both deployment pipelines run `go test ./... before` building any container image. The ordering is intentional: a failing test stops the pipeline while it is still free to stop, before images are pushed to a registry and before a rollout begins.

8 Local Deployment: a Self-Hosted GitOps Chain

The first deployment target reproduces, entirely on a single laptop, the chain that a team would normally rent from a provider: a Git server, a CI runner, an infrastructure-as-code tool, a hypervisor and a Kubernetes cluster. Nothing in it is a simulation — every component is the one that would be used in production, only hosted locally.

Gitea Actions → OpenTofu → Multipass → Ansible → Kubernetes (kubeadm)

8.1 The Five Layers

- **Gitea** — a self-hosted Git server, with an `act_runner` registered against it. It provides the same workflow syntax as GitHub Actions, so the pipelines are transferable.
- **OpenTofu** — declares the virtual machines: one control plane and one worker, their CPU, memory and disk, and the SSH key injected through `cloud-init`. It also renders the Ansible inventory from

the addresses Multipass assigns, so the two tools stay in step without manual copying.

- **Multipass** — the hypervisor providing Ubuntu virtual machines.
- **Ansible** — three playbooks: prerequisites on every node (containerd, kernel modules, disabled swap), `kubeadm init` plus the Flannel CNI on the control plane, and `kubeadm join` on the workers.
- **Kubernetes** — the resulting cluster, onto which the application manifests are applied.

8.2 Three Workflows, Three Responsibilities

The pipelines are separated by what they change rather than by convenience, and each is triggered by the paths it owns:

- `infra.yml` — provisions the virtual machines and bootstraps the cluster. Triggered by changes under `terraform/` or `ansible/`.
- `deploy.yml` — runs the test suite, builds the container images, distributes them to the nodes and applies the manifests. Triggered by changes under `k8s/`, `microservices/` or `frontend/`.
- `destroy.yml` — tears the cluster down. Manual trigger only, never automatic.

Changing a microservice therefore does not re-provision the cluster, and changing the cluster does not redeploy the application.

8.3 Distributing Images Without a Registry

Because the runner and the cluster nodes live on the same machine, the pipeline avoids a container registry entirely: it builds the images, exports them with `docker save`, copies the archive to each node over SSH and imports it into `containerd` with `ctr`. This is possible only because the nodes are long-lived and reachable — an assumption that does not survive the move to a managed cluster, as Section 9 describes.

8.4 State and Secrets

The OpenTofu state file is kept in the host's home directory rather than in the repository, so that it survives a fresh checkout. Runtime secrets — database passwords, the OpenSky credentials, the Grafana password — are stored as Gitea Actions secrets and turned into Kubernetes **Secret** objects by the pipeline at deployment time. The TLS certificates used for mutual authentication between services are not stored anywhere: they are regenerated by a script on every deployment, which is why the private keys never appear in version control.

9.1 What Was Substituted

Concern	Local	AWS
CI credentials	static SSH keys	OIDC, no stored keys
Infrastructure state	file on the host	S3, versioned and locked
Kubernetes	kubeadm on Multipass	EKS, managed control plane
PostgreSQL, Redis	StatefulSets	RDS, ElastiCache
Neo4j	StatefulSet	Neo4j Aura, external
Container images	imported with <code>ctr</code>	ECR
Secrets	Gitea secrets	SSM Parameter Store
Observability	<code>k8sadmin/</code> scripts	the same scripts

The last row is the most informative. Every other line is a substitution; the observability layer required none, because it was already addressing Kubernetes rather than the infrastructure beneath it.

9.2 Federated Identity Instead of Stored Keys

The repository is public, which rules out storing an AWS access key as a secret. Authentication uses **OpenID Connect**: GitHub issues a signed token describing the workflow’s origin, AWS exchanges it for credentials valid for about an hour, and the trust policy on the IAM role accepts tokens only from this repository. The single value stored on GitHub is the role’s ARN, which is an identifier and not a credential — knowing it grants nothing without a token AWS will accept.

9.3 Remote State

GitHub-hosted runners are destroyed after every job, so a state file on disk would not survive to the next run. The state lives in a private, encrypted and versioned S3 bucket, with concurrent runs prevented by OpenTofu’s native S3 locking. Versioning is not decoration: the state records the mapping between code and real resources, and losing it would mean rebuilding that mapping by hand.

9.4 Network Design

A VPC of 10.0.0.0/16 spans two availability zones, each divided into three tiers:

- **public** — holds that zone’s NAT Gateway, with a route to the Internet Gateway.
- **private application** — hosts the EKS nodes, which have no public address and reach the internet through their own zone’s NAT Gateway. This tier is a /20 rather than a /24 because the AWS VPC CNI assigns a real VPC address to every *pod*, not merely to every node.

- **private data** — hosts the databases and has *no default route at all*. They are reachable from inside the VPC and can reach nothing outside it.

Each zone has its own NAT Gateway and its own route table, so outbound traffic never crosses into the other zone and the loss of either one is survivable. An earlier revision shared a single gateway to halve the cost; it was abandoned because it made the second zone depend on the first, producing redundancy that worked in only one direction.

The security groups protecting the two databases carry **no egress rules at all**. Since security groups are stateful, replies to permitted inbound connections still flow, so the practical effect is a pair of databases that can answer but never initiate a connection.

A **gateway VPC endpoint for S3** is attached to the private route tables. Container image layers served by ECR are in fact fetched from S3, and traffic crossing a NAT Gateway is billed per gigabyte; routing it through the free endpoint removes that charge entirely, which the cost analysis later confirmed.

9.5 Data Layer

PostgreSQL runs on **RDS** with a synchronous standby in the second zone and automatic failover behind a single endpoint. Redis runs on **ElastiCache** as a replication group — a standalone cache node cannot span zones — with a primary and a replica. The address published for the application is the *primary endpoint*, which follows whichever node currently holds that role; publishing a node address instead would have worked until the first failover and then broken silently.

Neo4j has no managed equivalent on AWS and remains on **Neo4j Aura**, outside the account. Its credentials, together with the OpenSky ones, are written to Parameter Store by hand: they belong to external services that OpenTofu does not create, and keeping them out of the tool also keeps them out of the state file.

9.6 Where Each Secret Lives

The dividing line is *bootstrap versus runtime*, not AWS versus external:

- **GitHub Secrets** hold only what is needed to *reach* AWS — the role ARN and the state bucket name.
- **Parameter Store** holds every runtime value: endpoints, database credentials, the Grafana password, and the credentials of both external services.

A consequence worth having is that the deployment pipeline could be replaced without moving a single secret.

9.7 Observability

Metrics Server and `kube-prometheus-stack` are installed by a dedicated workflow, kept separate from the application deployment because the Helm release changes far less often than the code. The two installation scripts are those written for the local cluster, **used unchanged**. The `ServiceMonitor` objects that tell Prometheus where the instrumented services publish their metrics are also the same, and were verified to be actively scraped on the cloud cluster.

The node count was raised from two to three for a reason worth recording, because it is not the obvious one. Memory was never the constraint — the whole monitoring stack requests roughly 640 MiB against more than 2 GiB free. *Pods* were: each node admits eleven, a ceiling imposed by network interfaces rather than by resources, and two nodes left exactly the number the monitoring stack needed, with no margin for uneven scheduling.

9.8 Constraints Imposed by the Account Plan

Three components of the reference architecture could not be built. Each was confirmed by an explicit error rather than assumed, and in one case verified twice.

- **Load balancer.** `OperationNotPermitted`: this AWS account currently does not support creating load balancers. Observed first from the Kubernetes service controller, then reproduced from the console on an unrelated VPC — which excludes any fault in the project’s own configuration.
- **CloudFront.** `AccessDenied`: your account must be verified before you can add new CloudFront resources. The configuration is written and remains behind a variable defaulting to false.
- **API Gateway and Cognito.** Reaching a private cluster from API Gateway requires a VPC Link, which requires the load balancer above. Cognito is a separate matter: it would duplicate the *User Manager*, which is already this system’s identity authority.

A fourth limitation is not a restriction but an observation: **OpenSky does not answer requests originating from AWS**. The same host replies in a quarter of a second from a residential connection and times out from inside the cluster, while other destinations answer normally from the same pods — the signature of a filter on cloud-provider address ranges. It does not prevent the deployment, since flight collection is only triggered once a user registers an interest.

9.9 Consequences for the Entry Point

Without a load balancer the cluster has no public address, and the application is reached through an authenticated tunnel established by `kubectl port-forward`. This is not a smaller load balancer: the

tunnel is opened through the EKS API server, requires valid IAM credentials and an access entry on the cluster, and exists only while the command runs.

The choice was made deliberately rather than worked around. Moving the nodes into public subnets and exposing a node port would have restored a public URL and even reduced the cost, but it would have dismantled the private network tiers, widened the reach of a compromised node to the database subnets and made the address change whenever a node was replaced. Given that the system's correct behaviour had already been demonstrated end to end, that trade was not worth making.

9.10 Cost Model and Operational Discipline

The binding constraint proved to be *uptime*, not architecture. With everything running the environment costs roughly \$0.35 per hour: about a dollar for a working session followed by a teardown, but some eight dollars for a day left running by mistake.

Three practices follow from this, and they shaped the order in which the work was done:

1. A cost budget with e-mail alerts was created *before* any billable resource. It is configured to exclude promotional credits, since a budget that counts them reports nearly zero while the credit silently drains.
2. The destroy workflow was written *before* the deploy workflow.
3. A verification script queries each service API directly after a teardown. The console's resource group is an eventually consistent index that keeps listing deleted resources for hours, and so cannot answer the question; the script also looks for resources OpenTofu cannot know about, such as a load balancer created by Kubernetes itself.

Measured spending over the whole development was under two dollars, of which the NAT Gateways were by far the largest share.

References

- [1] M. Schäfer, M. Strohmeier, V. Lenders, I. Martinovic, and M. Wilhelm, “Bringing Up OpenSky: A Large-scale ADS-B Sensor Network for Research,” in *Proceedings of the 13th IEEE/ACM International Symposium on Information Processing in Sensor Networks (IPSN)*, IEEE/ACM, Apr. 2014, pp. 83–94.